

# From KV Random IOPS to Layer Streaming: A Systems Model for SSD-Resident Weights with $O(1)$ Recurrent State

[Author Name]

## Abstract

Commodity SSD offload is often discussed as though large-model inference fails simply because NAND is slower than HBM. This paper argues for a narrower systems explanation: the decisive issue is the access pattern imposed by the model. Transformer decode on block storage is hostile because growing key-value state pushes the storage path toward queue-depth-1 random reads, whereas  $O(1)$  recurrent-state models restore a sequential layer-streaming regime. We formalize that distinction with a simple decode-time model, identify the critical batch point at which compute begins to hide storage, and discuss how mixture-of-experts routing can preserve or destroy sequentiality. Evaluation combines analytical bounds with a calibrated storage model derived from an io\_uring-style SSD path. On a representative Gen4 TLC configuration, effective sequential bandwidth reaches 5.85 GB/s at queue depth 1 and 6.56 GB/s by queue depth 4, while the random 4 KB regime remains dominated by per-read latency. The contribution of the paper is therefore not a claim of end-to-end production throughput, but a systems model that explains when SSD-native inference is structurally implausible and when it becomes analytically defensible.

## 1 Introduction

Large-model inference is usually framed as a memory-capacity problem: HBM is fast, SSD is slow, therefore SSD-resident inference is doomed. That framing is too coarse. The more important question is what the storage system is being asked to do. Block devices tolerate large sequential reads far better than fine-grained, low-queue-depth random lookups. If a model architecture forces the latter, SSD-native decode fails for structural reasons. If the architecture permits the former, the problem becomes one of bandwidth, overlap, and careful accounting.

This distinction matters because the cost gap between accelerator memory and flash remains extreme. For many research and hobbyist deployments, terabytes of flash are economically realistic while large HBM footprints are not. The relevant question is not whether SSD can emulate HBM. It cannot. The question is whether a different model family can expose an SSD-friendly access pattern.

The paper makes three contributions:

1. It formalizes why Transformer decode is structurally hostile to block storage and why  $O(1)$  recurrent-state decode changes the dominant I/O path from random lookup to sequential layer streaming.
2. It introduces a simple but useful storage-native timing model, including a critical batch point that explains when storage and compute begin to balance.
3. It provides a calibrated storage-side evaluation that separates what is analytical from what is simulated, instead of collapsing all quantities into a single end-to-end throughput headline.

## 2 Background and Problem Statement

Autoregressive Transformers carry forward a key-value cache whose effective footprint grows with sequence length [1]. Even when concrete implementations differ in caching behavior, page layout, or prefetch policy, the structural problem remains: long-context decode increasingly depends on revisiting prior context rather than advancing through a bounded recurrent state. On block storage, that trend pushes the system toward the least favorable operating point.

Let  $T_{\text{random}}$  denote the latency of a low-depth random 4 KB read. A first-order upper bound for per-token storage time can be written as

$$T_{\text{kv}} = L \cdot n_h \cdot T_{\text{random}}, \quad (1)$$

where  $L$  is sequence length and  $n_h$  is the number of heads or equivalent accesses per layer. The exact constants are implementation-dependent, but the asymptotic implication is robust: as  $L$  grows, the storage path becomes dominated by repeated small lookups.

O(1) recurrent-state models such as Mamba and RWKV replace the unbounded cache with a fixed-size state update

$$h_t = f(h_{t-1}, x_t, \theta). \quad (2)$$

The main systems consequence is not merely asymptotic elegance. It is that the dominant external traffic shifts back to streamed model weights plus a bounded recurrent state. This changes the storage problem from low-depth random IOPS to sequential bandwidth.

## 3 Storage-Native Timing Model

For a layer-streaming recurrent design, a useful first-order model is

$$T_{\text{layer}} = \max(T_{\text{io}}, T_{\text{compute}}), \quad (3)$$

with

$$T_{\text{io}} = \frac{W_{\text{layer}}}{BW_{\text{ssd}}} \quad (4)$$

and

$$T_{\text{compute}} = \frac{2 \cdot W_{\text{layer}} \cdot B}{FLOPS_{\text{gpu}}}, \quad (5)$$

where  $W_{\text{layer}}$  is the layer payload,  $BW_{\text{ssd}}$  is effective SSD bandwidth, and  $B$  is batch size. The model is intentionally simple, but it captures the main question for SSD-native inference: once weights are streamed rather than randomly chased, is the system limited by storage or by compute?

Equating the two terms yields an idealized critical batch size,

$$B^* = \frac{FLOPS_{\text{gpu}}}{2 \cdot BW_{\text{ssd}}}, \quad (6)$$

which marks the approximate transition between I/O-bound and compute-hidden regimes. The equation is not a universal operating rule, but it provides a useful way to reason about when larger batches can amortize the storage path.

Table 1: Representative calibrated sequential-read configurations from the io\_uring simulation model.

| Config           | QD | Raw BW (GB/s) | Effective BW (GB/s) | Bottleneck     |
|------------------|----|---------------|---------------------|----------------|
| Gen4 TLC         | 1  | 6.03          | 5.85                | PCIe/NAND path |
| Gen4 TLC         | 4  | 6.79          | 6.56                | PCIe/NAND path |
| Gen5 TLC         | 1  | 10.33         | 9.82                | NAND           |
| Gen5 TLC         | 4  | 11.64         | 10.99               | NAND           |
| Azure Lasv4-like | 1  | 4.01          | 3.93                | NAND           |
| Azure Lasv4-like | 4  | 4.51          | 4.41                | NAND           |

### 3.1 MoE and Access Regularity

Mixture-of-Experts models complicate the picture because sparse routing can reintroduce randomness. The cleanest storage-side mitigation is stripe batching: group tokens by routed expert so that expert weights are fetched in larger sequential sweeps rather than scattered micro-reads. This does not eliminate routing overhead, but it preserves the main insight of the paper: SSD-native inference is only plausible when the runtime protects sequentiality as a first-class invariant.

## 4 Evaluation Method

The paper intentionally combines two evidence classes:

- **Analytical:** asymptotic reasoning and the timing model above.
- **Calibrated simulation:** storage-path behavior derived from an io\_uring-style path model, including queue depth, effective NAND bandwidth, and per-command overhead.

This paper does not claim a full end-to-end 70B deployment. It also does not claim that every runtime optimization multiplies independently. Instead, it asks a narrower question: when the dominant weight path is sequential, what kind of storage regime becomes available?

## 5 Results

Table 1 summarizes the calibrated storage-side results used in this paper. The important point is not a single absolute number, but the shape of the regime change: sequential bandwidth saturates quickly, while the random-read regime remains latency-dominated.

Three observations follow.

**Sequential saturation arrives early.** On the representative Gen4 path, moving from queue depth 1 to queue depth 4 raises effective bandwidth from 5.85 GB/s to 6.56 GB/s. This suggests that a carefully engineered sequential runtime does not need extreme queue depth to reach most of the available storage bandwidth.

**Storage remains a bandwidth problem, not an HBM problem.** Even the strongest sequential regime remains far below accelerator-memory bandwidth. The claim of this paper is therefore not that SSD can replace HBM transparently. The claim is narrower:  $O(1)$  recurrent-state models expose a storage regime that is at least analyzable in terms of bandwidth rather than destroyed by random access.

**The right baseline for Transformers is not sequential throughput.** A low-depth random 4 KB regime incurs on the order of tens of microseconds per read. Any design that requires thousands of such reads per generated token is immediately hostile to SSD. This is why the access-pattern shift is the main contribution of the paper rather than a secondary optimization.

## 6 Discussion

The model explains why many offload systems succeed as prefill or residency mechanisms yet remain fragile at decode time. It also clarifies why architecture and runtime cannot be separated in SSD-native inference. The hardware does not merely need a faster I/O library; it needs a workload that respects the physical strengths of flash.

The model also helps discipline later optimizations. Compression, micro-pipelining, and multi-token prediction are only meaningful after the base schedule has been defined under one bandwidth budget. This paper therefore serves as the foundation on which later papers in the project build.

## 7 Limitations

This paper makes a structural claim, not a full-system deployment claim. In particular:

- it does not report measured end-to-end token throughput for a 70B recurrent model,
- it uses a calibrated storage model rather than raw device traces for every platform,
- it does not claim that MoE routing locality is guaranteed in all workloads,
- and it does not remove host-memory transfer costs from the path.

These limitations are deliberate. The paper is meant to establish the correct systems question before claiming a full implementation answer.

## 8 Related Work

The closest prior systems are SSD- or NVMe-backed offload frameworks such as FlexGen and DeepSpeed-Inference, which showed that flash can serve as a practical weight tier for Transformer systems [4, 5]. Empirical studies such as CHEOPS characterize how these frameworks exercise NVMe in practice [6]. The architectural contrast in this paper comes from  $O(1)$  recurrent-state models such as Mamba and RWKV [2, 3]. On the decode side, KV-cache systems and cache-compression work motivate why growing context is such a poor fit for block storage [7].

## 9 Conclusion

SSD-native inference should not be treated as a monolithic yes-or-no question. The decisive question is whether the model forces the storage subsystem into a random-IOPS regime or allows it to operate in its natural sequential-bandwidth regime.  $O(1)$  recurrent-state models do not solve the memory wall by themselves, but they transform the storage-side problem into one that can be analyzed, optimized, and, eventually, implemented with disciplined expectations.

## References

- [1] Ashish Vaswani et al. Attention Is All You Need. In *NeurIPS*, 2017.
- [2] Albert Gu and Tri Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. 2023.
- [3] Bo Peng et al. RWKV: Reinventing RNNs for the Transformer Era. 2023.
- [4] Zhen Zheng et al. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. 2023.
- [5] Reza Aminabadi et al. DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. In *SC Workshops*, 2022.
- [6] Zebin Ren, Krijn Doekemeijer, Tiziano De Matteis, Christian Pinto, Radu Stoica, and Animesh Trivedi. An I/O Characterizing Study of Offloading LLM Models and KV Caches to NVMe SSD. In *CHEOPS*, 2025.
- [7] Yuhan Liu et al. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *SIGCOMM*, 2024.